

GUÍA DE ESTUDIO

Laboratorio 7

RAG sobre Vertex AI Gemini, con página de chat — cómo orientarte en los archivos sin perderte

En una frase: una página web de chat → backend FastAPI → busca fragmentos relevantes en un índice vectorial local → le pasa ese contexto a Gemini → Gemini responde citando de qué documento sacó cada dato.

Basado en: `README.md`, `CONCEPTOS.md`, `INSTRUCCIONES.md` y el código en `src/` del lab.
Referencia personal de estudio — no reemplaza leer el código.

1. Cómo está organizado el lab

Todo vive dentro de `lab7/`. Siete carpetas/archivos raíz, cada uno con un rol claro:

Carpeta / archivo	Para qué sirve
<code>README.md</code>	Punto de entrada: qué hace el lab, mapa de contenido, quickstart.
<code>CONCEPTOS.md</code>	La teoría: qué es RAG, embeddings, similitud de coseno, chunking. Léelo antes que el código.
<code>INSTRUCCIONES.md</code>	Paso a paso operativo: credenciales GCP, correr local, desplegar, troubleshooting.
<code>src/</code>	Todo el backend en Python (7 archivos). El corazón del lab — sección 3 de esta guía lo desglosa.
<code>web/</code>	Frontend plano: <code>index.html</code> + <code>app.js</code> + <code>style.css</code> . Sin frameworks.
<code>data/docs/</code>	5 documentos <code>.md</code> ficticios ("Programa Helios") que el RAG consulta como base de conocimiento.
<code>data/index/</code>	Se genera al ingerir (embeddings + chunks). No existe hasta que corres <code>src/ingest.py</code> .
<code>tests/</code>	Pruebas de lo que NO depende de GCP: chunking y búsqueda por similitud.
<code>infra/</code>	Terraform opcional para desplegar a Cloud Run como IaC.

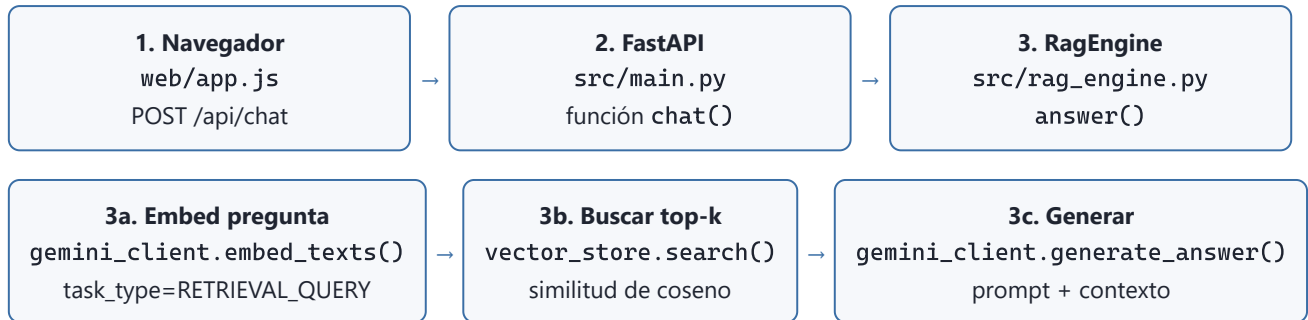
2. Orden de lectura recomendado

No empieces por `src/main.py`. Sigue este orden — cada paso te da el contexto que necesitas para el siguiente:

- 1 **README.md** — 5 min. Solo para tener el mapa mental completo y el diagrama de una línea.
- 2 **CONCEPTOS.md** — 15 min. Entiende *qué* hace cada pieza antes de ver *cómo* está codeada: RAG vs. fine-tuning vs. contexto largo, embeddings, `task_type`, similitud de coseno, chunking, el prompt.
- 3 **src/config.py** (61 líneas) — el más corto y sin dependencias de GCP en su lógica. Aquí ves qué variables de entorno existen y cuáles son sus defaults.
- 4 **src/chunking.py** (`chunk_text`) — parte texto en fragmentos. Sin llamadas a GCP, fácil de leer y de probar mentalmente.
- 5 **src/vector_store.py** (clase `VectorStore`) — el índice en memoria: guardar, buscar por similitud, persistir a disco.
- 6 **src/gemini_client.py** — el único archivo que habla con Vertex AI. Aquí están `embed_texts` y `generate_answer`.
- 7 **src/ingest.py** — conecta los tres anteriores: lee `data/docs/` → chunking → embeddings → guarda el índice.
- 8 **src/rag_engine.py** (clase `RagEngine`) — retrieval + arma el prompt + genera la respuesta. El "cerebro" del agente.
- 9 **src/main.py** — el servidor FastAPI que junta todo y expone `/api/chat`. Léelo al final: ahora cada línea tiene sentido.
- 10 **web/index.html + app.js** — el cliente que llama a `/api/chat` por `fetch()`. Rápido de leer, sin sorpresas.
- 11 **tests/** — confirma que entendiste chunking y similitud viendo qué se afirma en los tests.
- 12 **INSTRUCCIONES.md** — cuando quieras correrlo de verdad (local o Cloud Run).

3. Flujo de datos de una pregunta

Esto es lo que pasa, en orden, cuando alguien escribe una pregunta en el chat con RAG activado:



La respuesta viaja de vuelta: Gemini → RagEngine (arma RagAnswer con fuentes) → FastAPI → JSON → navegador.

Detalle que se pasa fácil por alto: el `task_type` al pedir un embedding debe ser `RETRIEVAL_DOCUMENT` cuando se ingesta un documento y `RETRIEVAL_QUERY` cuando se embebe la pregunta del usuario. Usar el equivocado no da error — solo empeora la búsqueda en silencio. Ver [gemini_client.py](#).

4. Referencia rápida: src/ archivo por archivo

Archivo	Qué exporta / hace	Depende de GCP
config.py	Settings (dataclass) + variable global settings. Lee GCP_PROJECT_ID, GCP_LOCATION, EMBEDDING_MODEL, GENERATION_MODEL, RAG_TOP_K del <code>.env</code> . Falla rápido y claro si falta GCP_PROJECT_ID.	No
chunking.py	<code>chunk_text(text, chunk_size=180, overlap=40)</code> — parte un texto en fragmentos de ~180 palabras con 40 de traslape.	No
vector_store.py	Chunk, SearchResult, clase VectorStore con <code>search(query_embedding, k)</code> , <code>save()</code> , <code>load()</code> , <code>exists()</code> . Todo en NumPy, en memoria.	No (solo NumPy)
gemini_client.py	<code>embed_texts(texts, task_type)</code> , <code>generate_answer(prompt)</code> , <code>generate_answer_with_usage()</code> , <code>count_tokens(text)</code> . Único punto de contacto con Vertex AI.	Sí
ingest.py	<code>load_documents(docs_dir)</code> , <code>build_index(docs_dir, index_dir)</code> — orquesta docs → chunks → embeddings → índice guardado. Se corre con <code>python -m src.ingest</code> .	Sí (vía gemini_client)
rag_engine.py	RagAnswer, clase RagEngine con <code>load()</code> , <code>retrieve(question, k)</code> , <code>answer(question, use_rag)</code> , y SYSTEM_PROMPT (la instrucción anti-alucinación).	Sí (vía gemini_client)
main.py	App FastAPI. <code>lifespan()</code> construye el índice si no existe al arrancar. Rutas: GET <code>/api/health</code> , POST <code>/api/chat</code> (función <code>chat()</code>), y sirve <code>web/</code> como estático.	Indirectamente
count_tokens.py	Script CLI: <code>python -m src.count_tokens "pregunta"</code> . Útil para estimar costo antes de llamar al modelo.	Sí

5. Correrlo en local — comandos mínimos

```
cd lab7
python -m venv .venv && source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env # editar GCP_PROJECT_ID adentro
gcloud auth application-default login

python -m src.ingest # construye el índice (opcional, main.py lo hace solo)
uvicorn src.main:app --reload --port 8080
# abrir http://localhost:8080
```

Detalle completo, credenciales GCP y despliegue a Cloud Run: ver `INSTRUCCIONES.md`.

6. Cómo comprobar que el RAG realmente aporta algo

La forma más directa de *ver* qué hace RAG (más que leerlo). Prueba en este orden, en la página de chat:

- 1 RAG activado** — pregunta algo que **sí** está en los documentos:
"¿Cuál es el umbral de reconciliación para columnas numéricas?"
Debería responder "0.01%" citando `03_proceso_validacion.md`.
- 2 RAG desactivado** — la misma pregunta. Gemini no tiene forma de saberlo (el Programa Helios es inventado) — debería admitir que no lo sabe o dar algo genérico.
- 3 RAG activado** — pregunta algo que **no** está en los documentos (ej. "¿de qué color es el logo de Acme Analytics?"). El `SYSTEM_PROMPT` en `rag_engine.py` le pide admitir que no lo sabe en vez de inventar.

7. Glosario exprés

Término	En una línea
RAG	Retrieval-Augmented Generation: buscar fragmentos relevantes primero, generar la respuesta usando solo esos fragmentos después.
Embedding	Vector de números que representa el significado de un texto; textos parecidos → vectores cercanos.
Chunking	Partir un documento largo en fragmentos pequeños antes de generar sus embeddings, para que cada uno tenga un significado más "enfocado".
Similitud de coseno	Mide qué tan parecidos son dos vectores (1.0 = iguales, 0.0 = sin relación). Con vectores normalizados, es solo el producto punto.
task_type	Le dice al modelo de embeddings si el texto es un documento a guardar o una pregunta a buscar — cambia el vector resultante.
top-k	Cuántos fragmentos más similares se recuperan para armar el contexto (<code>RAG_TOP_K</code> , default 4).
ADC	Application Default Credentials — cómo el SDK de Google se autentica en local sin API key, vía <code>gcloud auth application-default login</code> .

8. Checklist de autoevaluación

Si puedes responder que sí a todo esto, ya entendiste el lab:

- ☐ Puedo explicar, sin ver el código, qué hace cada uno de los 7 archivos de `src/`.
- ☐ Sé por qué `gemini_client.py` es el único archivo que toca Vertex AI, y por qué eso importa.
- ☐ Entiendo qué pasa si `task_type` está mal puesto (falla en silencio, no con un error).
- ☐ Puedo dibujar de memoria el flujo: navegador → FastAPI → RagEngine → (embed, buscar, generar) → Gemini → respuesta.
- ☐ Sé dónde vive la instrucción anti-alucinación y por qué sin ella el modelo inventa más.
- ☐ Corrí el lab local y vi la diferencia entre RAG activado/desactivado con mis propias preguntas.
- ☐ Sé qué pieza cambiaría si esto creciera a producción (Vertex AI Vector Search, pipeline de reingesta, multi-agente) — ver la última sección de `CONCEPTOS.md`.

Guía generada como apoyo de estudio a partir de README.md, CONCEPTOS.md, INSTRUCCIONES.md y src/ del lab7. Verifica siempre contra el código fuente.